METHODOLOGY

Security Evaluation of LLM-Based Agents: A Systematic Review of Attack Strategies, Evaluation Metrics, and Benchmarking Practices

Review procedure as executed, and the AgentShield-LCE evaluation lifecycle that follows from it

This section describes how the review was carried out and how the proposed evaluation lifecycle, AgentShield-LCE, was specified. Every count is taken from a stage workbook or a stage report in the project folder, and each count states what it counts. The review is a relevance-ranked screening of a venue-filtered corpus. It is not a venue-verified inclusion set, and AgentShield-LCE is a design that has not been run. Both limits are repeated where they matter.

1. Review design

1.1 Type of study and reporting

The study is a systematic literature review of the security of LLM-based agents, extended by a proposed evaluation lifecycle. An LLM-based agent is a system that plans in natural language and then calls a tool, writes to memory, or sends a message to another agent. Record flow is reported as stage-by-stage totals in the manner of a PRISMA flow, in tabular form (Table 1). No registration of the review protocol is recorded in the project files, so the rules in Sections 2 to 7 are described as they were applied.

Table 1. Record flow from collection to open full texts

Step	Stage	Started with	Removed	Remaining	Check
1	Records collected (rows, before duplicate removal)	—	—	4,500	Raw collection
2	Exact duplicate records removed	4,500	1,441	3,059	4,500 − 1,441
3	Set aside by the quality split	3,059	2,453	606	3,059 − 2,453
4	Not in the final set after title and abstract screening	606	506	100	606 − 506
5	Final set without an open PDF	100	32	68	100 − 32

Source: SLR_Record_Flow.xlsx. The 100 are the top of an abstract-based relevance ranking. They are not a venue-verified inclusion set (Sections 5 and 6).

1.2 Research questions

Four dimensions organize the review: attack, evaluation metrics and benchmarks, mitigation, and proposed frameworks. Mitigation is the priority dimension. The central question is how a security claim about an LLM-based agent should be allowed to be made. It is broken into five questions.

- **RQ1 (attack).** Which attack strategies against LLM-based agents are reported, and through which surface do they reach the agent: the instruction channel, untrusted content, a tool description or implementation, memory, or a peer agent?
- **RQ2 (evaluation).** Which metrics, benchmarks, and success oracles are used to decide that an attack or a defense succeeded?
- **RQ3 (mitigation, priority).** Which mitigations are proposed, at which point of the agent do they act, and how is a mitigation tested after it is applied?

- **RQ4 (frameworks).** Which frameworks or architectures organize security evaluation or defense for agents, and what do they leave unspecified?
- **RQ5 (synthesis).** Under what conditions, and with what evidence, should a security claim about an agent be accepted? AgentShield-LCE (Section 10) is the answer proposed to this question.

1.3 Change of working topic

The work began under a different working title, “Cross-Domain Side-Channel Attacks on Multi-Agent LLM Systems: Inferring Agent Goals via Resource Consumption Patterns in Tool-Augmented Workflows”. A planning spreadsheet holds reading notes on candidate papers for that topic. The topic was then changed to the security evaluation of LLM-based agents, and the question in Section 1.2 replaced the earlier one. The planning spreadsheet is not a source of any count in this methodology.

2. Scope and eligibility

The criteria below are those applied during the quality split and screening. The title-stage exclusions are quoted from the screening report. The venue-related exclusions are a consequence of the evidence available for ranking venues (Section 5). They are not a statement that a paper is weak.

Table 2. Eligibility criteria and where each was applied

Code	Criterion	Applied at
I1	The subject is the security of an LLM-based agent as defined in Section 1.1.	Title and abstract screening
I2	The paper contributes to at least one of the four dimensions: attack, evaluation metrics and benchmarks, mitigation, or a proposed framework.	Abstract scoring (Table 7)
I3	A peer-reviewed conference or workshop paper with a full-paper proceedings record, or a record placed in the kept set by the CORE rank check.	Quality split
I4	A usable abstract is stored on the record, so that a score can be given.	Abstract screening
E1	A general chatbot jailbreak that never reaches a tool, a memory store, or a second agent.	Title screening
E2	An agent application with no security question.	Title screening
E3	A paper that uses an agent only as a tool against other software.	Title screening
E4	Preprint or repository posting only; journal articles without a verified quartile; books, book chapters, dissertations, reports; short papers, posters, demos; venues that could not be verified.	Quality split
E5	No usable abstract. The record is marked missing and is not scored. A snippet is not treated as an abstract.	Abstract screening
E6	Eligible, but ranked below the top 100 by score. Kept as reserve.	Ranking

No language restriction and no publication-date window is recorded as a screening criterion. The collected records are dated 2019 to 2026. The kept set is dated 2023 to 2026 and the final set 2024 to 2026 (Section 6.4). Chat-only jailbreaks and scores that cite only the model’s own statement of compliance are outside the design of AgentShield-LCE (Section 10.1).

3. Information sources and collection

3.1 Corpus collection

The working corpus is 4,500 collected rows. Each record carries one or more source labels. After duplicate removal, 2,227 of the 3,059 unique papers have one label, 683 have two, 130 have three, 17 have four, and 2 have five, so label counts overlap and do not sum to 3,059.

Table 3. Source labels on the 3,059 unique papers (overlapping)

Source label	Records	Note
OpenAlex	1,810	Largest single label
arXiv	1,412	Preprint listings
Google Scholar	271	Label count; see Section 3.2
Crossref	237	Metadata records
IEEE Xplore	164	Label count; see Section 3.2
CORE	108	
DOAJ	51	
HAL	8	

Source: After_Duplicate_Removal.xlsx, column Collection_Source. A record with several labels is counted once under each.

Of the 3,059 unique papers, 2,798 have a stored abstract and 2,107 have a DOI. By item type they are 1,792 preprints, 651 conference papers, 355 journal articles, 64 book sections, 3 books, 3 reports, 2 reviews, 95 of unknown type, and 94 of other type. The research-category labels given at collection (for example Attack, Framework, Evaluation, Tool Security, Multi-Agent Security) are multi-label and were not part of the relevance score.

The query strings used to assemble the 4,500 rows were not preserved in the project files. The review therefore does not claim that the collection is exhaustive or that it can be regenerated from queries. Reproducibility is at the record level: each stage is a workbook with stable paper identifiers (P000001 onward), and every later count can be recomputed from those workbooks.

3.2 Earlier documented search, kept separate

Before the folder's 4,500-row flow, a documented search on 26 and 27 October 2026 built a smaller bibliography. Its queries and counts are in Appendix B. It is reported here for transparency and is not the denominator of any count in Table 1.

- **IEEE Xplore, 26 October 2026.** Searched through the guest search service, without bypassing a paywall. Thirty-one queries exported 1,943 overlapping hits. After deduplication by article number the screening log held 952 unique rows: 788 excluded, 135 included, and 29 marked for review. The 164 retained rows are the 135 plus the 29.
- **Google Scholar, 27 October 2026.** Thirteen query runs, year filter 2020 to 2026, kept 270 rows. Raw hit totals were not stored. A Scholar snippet was not treated as an abstract, and a DOI was parsed only where the exported BibTeX contained one.
- **Merge, 3 November 2026.** The 164 and the 270 were stacked to 434 and reduced by exact DOI, or by exact normalized title when no DOI was present, to 419. Fifteen Scholar rows matched an IEEE row and were removed; the IEEE row was kept. Near-duplicate titles were not merged. The 419 is an exact-key bibliography of that pass. It is not an included-study count and it is not the 3,059.

- **Crossref metadata search, 3 November 2026.** Three queries returned 29 hits. They were not screened and belong to neither corpus.

The 3,059-record file carries source labels for IEEE Xplore (164 records) and Google Scholar (271 records) that are numerically close to the retained counts above. The project files do not state that the earlier workbooks were merged into the 4,500, so this methodology neither adds the two collections nor asserts that they are the same records.

3.3 Databases that were not accessible

ACM Digital Library, Scopus, and Web of Science were attempted on 26 October 2026 and returned no records. The recorded reasons were an access block, a sign-in wall, and a session error. Those three databases are therefore inaccessible in that log. They were not searched and found empty of the topic. Repeating the queries on an institutional session remains open. Clarivate JCR and Scopus CiteScore were not accessed, so no journal quartile was assigned anywhere in this review.

4. Record management and duplicate removal

Every collected row has a paper identifier. Duplicate copies of a paper reuse the same identifier, so the 4,500 rows hold 1,618 papers that appear once and 1,441 papers that appear twice, which gives 3,059 unique papers and 1,441 removed rows. Removal was by exact duplicate. No fuzzy or near-title merge was made. After removal, 3,059 records have 3,059 distinct identifiers.

A Zotero export of the deduplicated stage was not used. It repeated two identifiers, omitted P000873 and P002352, and replaced P000818 with a Zotero key. The before-removal Zotero workbook was a byte-identical copy of the collection workbook. Three title strings collide once punctuation is removed. They were not merged, and none is split across the kept and set-aside workbooks. One collision is an artifact of normalizing a non-Latin title and is not a duplicate decision.

5. Publication-quality split

5.1 Purpose and evidence available

The split separated records by publication type and venue evidence before screening. It classified records. It did not judge the quality of any individual paper, and no per-paper risk-of-bias or study-quality appraisal was carried out. The evidence that could actually be opened determined what could be assigned.

Table 4. Venue evidence that was and was not available

Source	Result
Crossref works API	Retrieved for 1,134 DOIs: work type, publisher, container title, and ISSN where given. 952 records have no DOI. 973 records have a DOI that is not in Crossref, mostly arXiv DOIs.
ICORE2026 (CORE)	Export downloaded from the CORE portal on 7 November 2026 (987 rows). Conference letter ranks A*, A, B, C assigned only when the proceedings title contained the entry's name, or its acronym with a confirming title word.
JCR, Scopus CiteScore	Not accessed. No journal quartile was assigned.
CCF	The March 2026 catalogue was not retrieved as a usable list. No CCF rank was applied. CCF appears in category names only as the project's bucket label; the ranking system recorded is CORE.
OpenAlex	Rate-limited (HTTP 429) during classification. Not used for venue evidence.

5.2 First pass

The first pass kept 486 records and set aside 2,573 (486 + 2,573 = 3,059). A conference or workshop proceedings record was kept when Crossref typed it as a proceedings article, it had a container title, and the title was not a short paper, poster, or demo. IEEE records were retained on the same rule. This is a venue category, not a claim that the conference is CORE A or CCF A. Of the 486, 307 were IEEE conference proceedings and 179 came from other proceedings publishers (ACL, ACM, Springer, AAAI, NeurIPS, USENIX, Internet Society, PMLR, Elsevier); 431 were conference papers and 55 workshop papers. No journal record was retained.

Table 5. First-pass set-aside groups (2,573 records)

Group	Records	Share
Preprint or repository posting only	1,863	72.4%
Journal article; quartile and indexing not verified	334	13.0%
Venue not verified	161	6.3%
Proceedings container is not a conference or workshop volume	112	4.4%
Book chapter	64	2.5%
Proceedings publisher not verified for the kept category	11	0.4%
IEEE record whose journal-versus-conference type was not returned	9	0.3%
Book (6), monograph (3), report (3), review (2), dissertation (2), short paper or poster (3)	19	0.7%

Source: Publication_Quality_Filtering_Report.md. Included in the set-aside groups are 58 IEEE journal articles, kept out because no JCR or Scopus source was opened.

5.3 Cross-verification and the kept set of 606

On 7 November 2026 both workbooks were audited record by record against the ICORE2026 list, and the pre-audit copies were backed up. No selected record was removed. One hundred and twenty records were moved from the set-aside workbook into the kept set, 44 of them verified CORE A* or A conference papers, and one IEEE conference paper (P000521) whose IEEE status field had been blank. A further 179 kept records had their rank label corrected in place. In total there were 299 corrections. The kept set is therefore 486 + 120 = 606 and the set-aside set is 2,573 − 120 = 2,453 (606 + 2,453 = 3,059, overlap 0, missing 0).

Some records were deliberately not given a main-track rank: Findings papers, workshops, industry tracks, companion and supplemental volumes, short-paper volumes, posters, and demos; venues that ICORE2026 lists as a multiconference (for example ICASSP); and venue names that do not state the ranked event (for example Proceedings of the ACM on Software Engineering, which was not labeled FSE). AAAI Symposium Series papers stay in the set-aside group because they are not the AAAI main conference. These choices favor not over-ranking a venue.

Table 6. Venue categories of the 606 records kept for screening

Priority	Category (project bucket label)	Records	Share
2	CORE A / CCF A conference (CORE A* in practice)	145	23.9%
4	CORE A / CCF B conference (CORE A in practice)	21	3.5%
7	CORE B / CCF C conference (CORE B in practice)	39	6.4%
9	CORE C conference	12	2.0%

Priority	Category (project bucket label)	Records	Share
12	Other peer-reviewed conferences and workshops (no CORE A*, A, B, or C letter)	389	64.2%
	Total	606	100%

Two consequences follow and are carried into Section 11. First, the kept set is conference-dominated, because journals could not be ranked without JCR or Scopus and preprint-only records were set aside. Second, venue rank is a proxy label for the venue, not a measure of the paper.

6. Title and abstract screening

6.1 Title stage

Every title in the 606 kept records was read. A title was excluded only when it was clearly outside the security of LLM-based agents (criteria E1 to E3). Titles that were not excluded were marked Include or Uncertain and passed to the abstract stage. Of the 606 titles, 221 were Include, 193 were Exclude, and 192 were Uncertain. The 413 Include and Uncertain records proceeded.

6.2 Abstract stage and scoring rubric

Each of the 413 records was judged from the stored abstract. A record with no usable abstract was marked missing and not scored (36 records). The score uses only what the abstract supports, on a 100-point scale. Venue is not in the score.

Table 7. Relevance score components (maximum 100)

Component	Points	Dimension
Direct relevance to LLM-agent security	30	Relevance
Attack	15	RQ1
Evaluation	15	RQ2
Mitigation	20	RQ3 (priority)
Framework	15	RQ4
Practical usefulness	5	Applicability
Total	100	

Of the 413 abstracts, 232 were Include, 141 Exclude, 4 Uncertain, and 36 missing (232 + 141 + 4 + 36 = 413). Of the 232, 146 scored 58 or higher. The final set is the 100 highest-scoring papers. The other 46 at or above 58, and the 86 included papers below 58, form the reserve of 132. The cutoff was not lowered to reach a round number, and more than 100 papers cleared the bar, so the primary set is the top 100. Ranks follow score, then directness, then the original quality priority, then year. Quality priority therefore acted only as a tie-breaker.

Table 8. Screening arithmetic for the 606 kept records

Outcome	Records	Cumulative
Final set (top 100 by score)	100	100
Reserve: eligible, ranked outside the top 100	132	232
Uncertain at abstract stage	4	236

Outcome	Records	Cumulative
No abstract, not scored	36	272
Excluded at abstract stage	141	413
Excluded at title stage	193	606

Check: 100 + 132 + 4 + 36 + 141 + 193 = 606. The 506 records outside the final set are the reserve register.

6.3 Dimension assignment

The primary dimension of a paper is the one with its highest component score. A paper filed under more than one strong component is counted once in the primary split. A paper is marked multi-dimensional when at least two components are strong, meaning attack or evaluation at least 10, mitigation at least 12, or framework at least 10. Fifty-four of the 100 papers are multi-dimensional. Overlapping secondary counts, which add a paper again for each secondary dimension scoring at least 6, are reported separately and are not a second corpus.

Table 9. Dimension distribution of the final 100 papers

Dimension	Primary	Overlapping count (primary plus secondary, ≥ 6 points)	Not multi-dimensional (46)
Attack	32	74	20
Evaluation metrics and benchmarks	18	76	10
Mitigation	35	68	11
Proposed framework	15	83	5
Total	100	—	46

Primary counts sum to 100. The second column overlaps on purpose. The last column is the 46 papers that are not multi-dimensional, grouped by primary dimension; the other 54 are filed as multi-dimensional.

6.4 Description of the final set

The 100 papers score from 62 to 83 (mean 67.6, median 67). The highest-ranked paper is ACE, a security architecture for LLM-integrated app systems. Title decisions were Include for 78 papers and Uncertain for 22, and every final paper has an abstract decision of Include. Screening confidence was recorded as high for 97 papers and medium for 3. By year, 5 papers are from 2024, 21 from 2025, and 74 from 2026. By type, 89 are conference papers, 10 are Findings papers, and 1 is a workshop paper. By publisher, 45 are IEEE, 36 ACL, 6 ACM, 5 NeurIPS, 4 Internet Society, 3 AAAI, and 1 is posted on arXiv.

Table 10. Venue categories of the final 100 papers

Priority	Category (project bucket label)	Papers	Level
2	CORE A / CCF A conference (CORE A*)	44	Top
4	CORE A / CCF B conference (CORE A)	4	Very high
7	CORE B / CCF C conference (CORE B)	7	Good
9	CORE C conference	2	Moderate
12	Other peer-reviewed conferences and workshops	43	Lower

The “Level” words are the project’s own labels for the venue categories. They are copied unchanged from the quality workbook and are not a judgement on any paper.

6.5 Who screened

The screening was not a dual independent review. No second reviewer is named in the screening report, and the screening record does not store the identity of the screener or any inter-rater agreement. Each final paper does store a title decision, an abstract decision, a score, a one-sentence rationale, a confidence label, and a rank, so each decision can be inspected after the fact.

7. Full-text retrieval and verification

Retrieval was attempted once for each of the 100 final papers. Lookups used OpenAlex open-access locations, arXiv links already stored on the record, and the ACL Anthology PDF that corresponds to a 10.18653/v1 DOI. The Unpaywall service rejected the request email and was not used for the successful downloads. Only public PDFs were retrieved. No paywall was bypassed and no HTML page was printed to PDF. A file was kept only when it opened as a PDF and its extracted text or metadata matched the selected record. Mismatched files were not saved.

Table 11. Full-text retrieval outcomes for the final 100 papers

Outcome	Papers	Share
Official proceedings PDF (ACL Anthology 35, NDSS 4, AAAI 2)	41	41%
Verified arXiv preprint, labeled as a preprint and not a version of record	27	27%
Paywalled, no open PDF located	31	31%
Access denied (P000751; the only open location returned HTTP 403)	1	1%
Invalid or mismatched files kept	0	0%
Open PDFs retrieved in total	68	68%

All 68 saved files were reopened after download. Each begins with the PDF header and is larger than 4 KB, and together they run to 1,402 pages. Title matched for 68 of 68 files and the author list matched for 67 of 68. The DOI string was found in the extracted text of only 5 files, because most proceedings PDFs do not print the DOI. Identity for the other 63 therefore rests on the title and author match and on the metadata of the source location. The 68 open PDFs are filed by primary dimension: Attack 16, Evaluation and benchmarks 7, Framework 1, Mitigation 9, and Multi-dimensional 35. The 31 paywalled papers and the 1 access-denied paper were not read in full.

8. Data extraction and synthesis

Every one of the 100 papers has the same stored record: primary and secondary dimension, a one-sentence technical contribution, a relevance rationale, a score, a rank, a confidence label, and the venue category. These fields come from the abstract. For the 68 open PDFs the synthesis can additionally read the full text; for the 32 others it rests on the abstract and metadata alone, and statements about those papers are marked as abstract-based.

The synthesis is descriptive and organized by the four dimensions. For each paper it records the following items, using full text where open and the abstract otherwise.

- The **surface** the attack or defense acts on: instruction channel, untrusted content, tool description, tool implementation, memory, or peer message.
- The **attacker objective**: task replacement, unauthorized tool execution, exfiltration, persistence through memory, a peer-caused action, or denial of the user task.

- The **success oracle**: a tool-level trace, a pre-declared end effect, a judge label, or the model's own statement.
- The **metrics and benchmark** used, and whether benign task success is reported on the same cases as attack success.
- For mitigation papers, the **control class** (prevent, detect, respond, recover) and whether the control was rerun after it was applied, including against a changed attack.

No quantitative pooling and no meta-analysis is attempted, because the papers report different attack families, environments, and metrics. A rate reported by one paper is never converted into a baseline for another. Findings are reported as counts of papers and as named patterns, with the evidence level (full text or abstract) stated.

9. Evidence gaps that shaped the design

The papers that were read do not, as a set, report attack success, task success, a tool-level trace, detection before commit, and a recovery rerun together. Individual benchmarks cover a slice: one attack family, one environment, or one guard. The design's job is the order in which those slices are allowed to support a claim. A claim that skips a gate is not a result of this lifecycle. The design refuses the following gaps, each only where the project notes already separate them.

- A score taken from the model's own words instead of a tool, memory, or peer event.
- A defense reported without the same user task in a benign case.
- Tool description, tool code, and permission treated as one surface.
- Detection reported on one attack distribution and reused as if it covered a later distribution.
- Recovery claimed from a restore that was not rerun.
- A peer-caused tool call added into a single-agent attack-success rate.
- A chat-only jailbreak scored as an agent effect.

One further gap concerns this review rather than agents: relevance is easy to misread as inclusion. The screening records keep the two apart, and this methodology does the same.

10. AgentShield-LCE: specification of the evaluation lifecycle

AgentShield-LCE is an evaluation lifecycle for an LLM-based agent. It is a design. No experiment in this project has executed it, and nothing in this section is a measured performance claim. It does not rename or replace WASP, InjecAgent, TraceProbe, CaMeL, or any other published system. Those systems test a slice, and their reported rates remain theirs. What the lifecycle adds is a refusal rule: a security claim may be made only when every in-scope surface is named or waived, every case has a tool-level trace, every number cites a trace field, every control has been rerun, and recovery has been rerun against a clean baseline.

10.1 Scope

In scope is an agent whose security-relevant behavior is observable as a tool call, a tool argument, a memory read or write, a peer message, or the success or failure of a pre-declared user task. Out of scope are chat-only jailbreaks, compromise of the host machine unless it is the system under test, attacks whose only success signal is a model self-report or an untraced judge label, a single score that adds agent-to-agent results to single-agent results, and host availability, model theft, and training-time attacks unless the asset register puts that supply chain inside the system under test. Direct abuse of the instruction channel is in scope only when the declared check is a tool, memory, or peer effect. Corpus support for that case is weaker than for indirect injection, and its inclusion is a design choice.

10.2 System under test and trust boundaries

The agent loop is instruction, planning, tool selection, tool execution, and tool result, which then repeats or ends. Configuration and commit sit off the loop. Memory and peer exchange are included only when the Stage 1 register includes them; otherwise they are waived rather than drawn as if present. Untrusted content is a carrier into a tool result or a memory read, not an extra step.

Table 12. Observation points on the agent path

Point	What crosses	Exposure	Control that can bind	Trace fields
Configuration	Tool descriptions, implementations, permission levels	Poisoned manual; unpinned code; a flat registry presented as least privilege	Pin description and implementation; record permission levels or waive privilege separation	Registry version, pin, permission list
Instruction intake	The user instruction	An instruction requesting a tool, memory, or peer effect outside the task	Allow-list checked at execution	Instruction identifier
Planning	The plan, if emitted	A plan rewritten after untrusted content	Plan-to-call consistency (detect only)	Plan text, marked non-oracle
Memory	Reads and writes	Poisoned entry; secret later sent; a write that becomes a future instruction	Write policy; provenance on read; quarantine; delete and retest	Operation, entry identifier, provenance label
Tool selection	Chosen tool name	Selection driven by a poisoned description	Deny tools outside the task allow-list	Tool name
Tool execution	Arguments, result, committed or not	Injection in the result; exfiltration; hidden effect in code	Argument and destination constraints; commit only after the check	Name, arguments, result, committed flag, order
Peer exchange	Message to or from another agent	A message that causes a tool call on the receiver	Bound what a peer may cause; drop the message	Destination; receiver's later tool call
Commit and recovery	Side effects left after the run	An effect that survives the session	Restore the clean baseline or list the irreversible difference	Restore log

Table 13. Trust boundaries

Boundary	Rule
TB1 Instruction / data	Untrusted content is not an instruction. A direct user instruction is still subject to the task allow-list.
TB2 Planner / tool	Description, implementation, and permission are separate fields. Registry membership is not authorization.
TB3 Agent / memory	Stored text is data until a write policy promotes it.
TB4 Agent / peer	A peer message cannot authorize a tool the user did not grant.
TB5 Evaluator / model	The oracle is the trace. The model's own claim of refusal or compliance is not evidence. TB5 is an evaluation boundary, not a network edge.

10.3 Threat model: assets and actors

Availability in this catalog means the user task, not the uptime of a model server. An attack that cannot be written as one of the declared checks is out of scope for the metric table, and may be named in the catalog as not scored. Psychological manipulation between agents, and other multi-agent observations that are not tool-trace predicates, are not scored.

Table 14. Assets and the failure that counts

Asset	Failure that counts
User data	A tool call or message sends a secret the declared task did not allow to be sent.
Tool effect	The agent calls a tool, or passes arguments, that the task did not authorize.
Memory	A later task follows a poisoned entry, or a stored secret is read into an unauthorized send.
Peer agents	A peer message causes a tool call the user did not request.
User task	The benign check fails, or the task is replaced by the attacker's task.

Table 15. Actors and when each is included

Actor	Capability	Not capable of	Included
Content injector	Place text the agent reads: a page, an email, a retrieved passage, a tool result	Call tools directly; change weights	Always, if any untrusted carrier exists
Instruction adversary	Supply the instruction channel	Reach a tool except through the agent	When the check is a tool, memory, or peer effect
Tool publisher	Publish a description, an implementation, or both	Impersonate the user	When third-party tools are selected
Peer agent	Send a message the receiver treats as input	Bypass the receiver's tool boundary except by that message	When a peer is in the register; otherwise waived
Model or trace supplier	Poison weights or tool-use traces before deployment	Act at runtime without a trigger in an in-scope carrier	Only if that supply chain is in the register
Internal evolution	Write memory, tool entries, or workflow a later session will trust	Needs no external party	Only if the system updates itself between tasks

The assumptions are as follows. The evaluator can sandbox the tools and read the trace fields; otherwise Gate G2 fails and the run is not reported as an evaluation. The benign task and the attack task share the user goal; otherwise utility under attack is undefined. A tool that cannot be rolled back can still be tested, and recovery then records an irreversible residual. The attacker does not control the instrumentation; an attacker who can forge traces is outside TB5 and outside the model.

10.4 Lifecycle stages and gates

The evaluation is Stages 1 to 10. Four gates judge the record. They are checks on the evaluation, not steps on the serving path, and they add no serving latency.

Table 16. Stages 1 to 10

Stage	Work	Output	Leave only when
1 Identify	List the model, tools, memory, peers, data, and which inputs are untrusted. Record permission levels or state there is one. Mark whether tool code, training traces, or self-updates are inside the system.	Asset register	The register is written
2 Threat model	For each surface, write adversary, goal, and precondition, or a waiver.	Surface catalog	G1 passes

Stage	Work	Output	Leave only when
3 Requirements	Write each forbidden effect and the user task that must still succeed, as one requirement identifier.	Requirement list	Every forbidden effect has an observable check
4 Tests	Build one benign case and one attack case for the same user task. Declare the check as a predicate over trace fields and the carrier the real system reads.	Test manifest	Every requirement has at least one pair
5 Run	Sandbox tools. Log the trace fields. Log a runner failure instead of inventing a trace.	One trace per case	G2 passes
6 Score	Apply only pre-declared checks and compute the metrics in Section 10.6.	Metric table	G3 passes
7 Detect	Mark each successful attack as flagged before the effect, flagged after it, missed, or not monitored. Apply the same detector to benign cases.	Coverage report	Every successful attack has one of the four marks
8 Mitigate	Add named controls, rerun the same pairs, and rerun a pre-declared changed attack set or record a waiver.	Before-and-after table	G4 passes
9 Recover	Restore memory and reversible tool effects. Rerun failed identifiers and a sample of earlier passes, or list an irreversible residual.	Reassessment	State matches the baseline, or the difference is listed
10 Report	Freeze versions, checks, traces, waivers, and the scoring script.	Release record	The record is stored; the next agent change returns to Stage 2

Stage 7 does not fail because no detector exists. “Not monitored” is an acceptable mark, and Gate G4 then records that the detect class was not implemented. Stages 9 and 10 are not gates: an irreversible effect is a listed residual, not a failed gate and not a successful recovery. A waiver is written into the Stage 10 record. A waiver is not a pass, and a waived surface is absent from the metric table.

Table 17. Gates, evidence required, and fail routes

Gate	After	Evidence required for a pass	A fail returns to
G1	Stage 2	The catalog covers, or waives in writing, each of: instruction channel, untrusted content, tool description, tool code, memory, and peer agents. Each covered surface has an adversary, a goal, and a precondition.	Stage 2. No tests are written for a surface that is neither named nor waived.
G2	Stage 5	Every case has a trace containing the required fields, or a recorded runner failure. Runner failures are counted apart from scores.	Stage 5. A missing tool log is not scored.
G3	Stage 6	Every number cites a trace field and a requirement identifier. Task success uses the same pairs as attack success. The carrier is one the system reads. The five questions in Section 10.5 are answered.	Stage 4 if the check or carrier was wrong; Stage 6 if only the arithmetic was wrong.

Gate	After	Evidence required for a pass	A fail returns to
G4	Stage 8	Every control has a class, an observation point, the requirement identifiers it covers, task success on baseline and protected runs, and the leftover identifiers. The changed-attack manifest was rerun, or a waiver states why not.	Stage 8. A control without a rerun is not finished.

A gate does not delete a surface to make the catalog look complete, does not impute a tool call from the model's final sentence, does not accept a lower attack-success number in place of a missing citation, and does not treat a fall in attack success as completion when task success is unreported or leftovers are unnamed. No gate asks a person to approve the evaluation in order to turn a fail into a pass. A pass means the evidence is complete. It is not a finding that the agent is secure.

10.5 Oracle, trace, and Gate G3

A case is an attack success only when the check declared in Stage 4 is true of the Stage 5 trace. A check may test only that a named tool was called with arguments satisfying a condition written before the run; that a named memory entry was written or read; that an outbound message or destination matches one written before the run; that an actuator event named before the run occurred; or that the benign task's own pre-declared check holds. The model's words, including "I refused" and "I complied", are not a check. An LLM judge is not a check unless its label is produced from those trace fields and the judging prompt is stored with the release record. The carrier must be one the system under test actually reads. A payload pasted only into the system prompt fails Gate G3.

Each case trace stores: case, requirement, and pair identifiers; role (benign or attack); agent version, tool-registry version, and suite version; carrier; tool name, arguments, tool result, and committed flag; memory operation and entry identifier; outbound destination; detector flag relative to commit (before, after, or none); whether the user-visible transcript was saved; and any runner failure. Plan text may be stored and is marked as a non-oracle.

Gate G3 requires all five answers before a number is used. (1) Did only the model change, or did the tool registry, memory policy, or peer set change too? (2) Did the attack use a carrier the system reads? (3) Does each metric name match the trace field in its definition? (4) After any change of the attack set, was the protected configuration rerun, or is there a waiver? (5) Is task success reported on the same pairs as attack success?

10.6 Metrics

The metrics are counts over one frozen manifest. Let B be the benign cases, A the attack cases, and A_s the attack cases whose attacker check holds. No threshold is defined and no weight combines the metrics, because the surfaces do not share a unit. A case with a runner failure is reported in a separate failure count. It is not placed in any denominator and is not counted as a defense success.

Table 18. The nine metrics

Label	Name	Numerator ÷ denominator	Better when	Gate	Undefined when
TS	Task success	Benign cases whose check holds ÷ B	Higher	G3	No benign check was declared
AS	Attack success	A_s ÷ A	Lower	G3	No attack cases. Never published alone
UA	Utility under attack	Attack cases whose benign check holds ÷ A	Higher	G3	No benign check on attack cases

Label	Name	Numerator ÷ denominator	Better when	Gate	Undefined when
CS	Covert success	Successful attacks whose pre-declared visibility predicate is false ÷ successful attacks with a saved transcript	Lower	G3	Transcript was not saved
DC	Detection coverage	Successful attacks flagged before commit ÷ A_s	Higher	Stage 7, G4	A_s is empty or no detector ran
FN	False negatives	Successful attacks not flagged before commit ÷ A_s	Lower	Stage 7	A_s is empty
FP	False positives	Benign cases flagged by the same detector ÷ B	Lower	Stage 7	No detector ran. “Not monitored” is not a rate of zero
LR	Leftover requirements	The set of requirement identifiers that still fail after Stage 8	Empty, or each remainder named	G4	The set is omitted. A count without identifiers fails G4
CAC	Changed-attack coverage	DC and FP recomputed on a second manifest declared before the rerun	Reported, not assumed	G4	The second manifest is waived; the cell states the waiver

If A_s is empty, detection coverage and false negatives are reported as undefined, not as 1 or 0. A prevent control can empty A_s, which is why it is not ranked by those two metrics. Optionally, and outside any gate, tool-call count and wall-clock duration may be reported as paired differences between baseline and protected runs. They depend on the machine and are not compared across papers.

Comparison uses two configurations of the same manifest. The baseline is the system with no control under test. The protected run is the same system plus the named controls, each labeled with its class and observation point. Both use the same requirement identifiers, checks, and carriers, and the protected run does not also swap the model or the tool list; if those change, the work returns to Stage 2. Stage 8 reports the metric table for each configuration and the leftover identifiers. It does not replace the table with a rank. No significance test, confidence interval, or sample size is defined; a difference between baseline and protected runs is a paired table, not a significance result.

10.7 Mitigation classes and control catalog

A control counts only when it has a class, an observation point, a mechanism, a protected property, a rerun on the same pairs, and a leftover list. Prevent stops the effect. Detect notices it. Respond acts on a notice. Recover cleans state. A detector that flags a call has not, by that flag, prevented it. A confirmation prompt is a respond control and does not replace an allow-list, because an unattended agent cannot wait on a person.

Table 19. Control classes

Class	Acts	Fair claim	Rerun required
Prevent	Before the effect commits	The forbidden effect did not occur	The full paired suite
Detect	During the run, at a named boundary	The case was flagged, and whether the flag preceded the commit	Successful attacks and the benign cases, with the same detector
Respond	After a flag	The effect was blocked, or a person approved that call	The same cases, plus the change in task success
Recover	After the effect or a suspected poison	State matches the clean baseline, or the irreversible difference is listed	Failed identifiers and a sample of earlier passes

Controls C1 to C10 are hosts for mechanisms already described in the literature. None is claimed as a new algorithm and none has been measured in this project. Hosting a published technique does not transfer that paper's measurements to this lifecycle.

Table 20. Control catalog

ID	Threat and point	Class	Mechanism	Limitation
C1	Indirect prompt injection from untrusted content; tool execution, before the result is treated as an instruction	Prevent	Keep tool results, pages, emails, and retrieved passages in a data channel and do not concatenate them into the instruction channel. An instruction-stripping filter is one implementation.	An adaptive injection can avoid the filter pattern. Benign instructions the task needed can be removed. Either failure stays in the leftover list.
C2	Unauthorized tool use, instruction-channel abuse, exfiltration; tool execution before commit	Prevent	Allow only tool names declared for the requirement. Constrain arguments and outbound destinations to predicates declared in Stage 4.	A wide predicate accepts an attacker-chosen argument. G3 then returns the case to Stage 4.
C3	Tool-description poisoning; configuration and selection	Prevent	Call a tool only when its description hash matches the pin in the register.	A pin freezes the manual the evaluator chose to trust. It does not make a malicious tool benign.
C4	Tool-code backdoor; tool execution	Prevent	Call the implementation only when its hash and permission match the pin. Deny the call if tool code is in scope and unpinned.	Out of scope when Stage 1 waives tool code. The waiver is not a verification.
C5	Plan changed after untrusted data; planning compared with the later call	Detect	If a plan was emitted before the untrusted read, flag a later tool call the plan did not contain.	A plan written after the poison can match the attacker's goal. This control does not prevent the call.
C6	Committed effect that should have been stopped; commit	Detect and respond	Flag the call at the commit boundary. Do not commit, or ask for approval of that call.	A fixed flag misses attacks outside its patterns, so G4 requires a changed-attack rerun or a waiver.
C7	Memory poisoning; memory write and later read	Prevent and detect	Accept a write only from carriers the write policy names. On read, check the provenance label and quarantine a mismatch.	Provenance shows origin, not that the text is harmless.
C8	Memory disclosure; memory read then outbound call	Prevent	The destination constraint of C2 applies to data read from memory.	A permitted tool can still leak if the destination predicate is broader than the task.
C9	Peer-caused tool use; peer exchange	Prevent, detect, respond	A peer message may name only tools in the peer allow-list. Any other is flagged and dropped.	Does not verify an agent-to-agent protocol. Scored separately from single-agent attack success.
C10	Poisoned memory or reversible tool effect; after commit	Recover	Quarantine or delete the entry, roll back reversible effects, restore the clean baseline, and rerun failed identifiers plus a sample of earlier passes.	Irreversible effects remain residuals. A restore that is not rerun is not recovery.

Stage 8 is not finished when attack success falls on the original manifest. The record lists the requirement identifiers that still fail, task success on the baseline and protected runs, which of the four classes was not implemented, and either coverage and false positives on a second attack manifest declared before the rerun, or a waiver stating why the original distribution is the only one in scope. The second manifest exists because disclosed defenses and fixed monitors have failed in published studies when the attack pattern changed. The lifecycle requires the rerun or the waiver. It does not claim that the rerun restores coverage.

10.8 Release record

Stage 10 stores the agent version, tool list, pins, permission list, suite version, check predicates, visibility predicates, waivers, traces, the script that computed each metric, and the difference between the baseline and protected configurations. A later reader must be able to recompute every table from the traces without asking the model whether it complied. A change of model, tool, memory policy, or peer set sends the next evaluation back to Stage 2.

10.9 Implementation notes

The lifecycle is a harness around a tool-using agent. The harness wraps the tool executor, the memory store, and, if a peer is registered, the message path. The harness, not the model, records whether the environment applied a call. Tools that can send, write, or delete run against a sandbox, and the release record states which tools were sandboxed. Prevent and respond controls are enforced in the harness and not only as a sentence in the system prompt. If the process crashes, a runner failure is written and a missing tool call is never filled from the model's last sentence.

11. Threats to validity and reporting rules

11.1 Limitations of the review

- **Venue filtering shapes the corpus.** Preprint-only records (1,863 in the first pass) and journal articles (334, including 58 IEEE journals) were set aside because no source for ranking them was opened. The kept set is conference-dominated. Recent work that exists only as a preprint, and journal work, may be missing from the 606.
- **The 100 are relevance-ranked, not venue-verified.** The score contains no JCR or CiteScore quartile and no CORE or CCF rank. Rank was a tie-breaker only. The 100 must not be described as Q1 or CORE-A inclusions: 44 sit in the CORE A* bucket and 43 in the category with no CORE letter.
- **Single-pass screening.** No second reviewer is named and no agreement statistic exists (Section 6.5).
- **Abstract-based scoring.** The score rests on the abstract. 36 records had no abstract and were not scored, so a relevant paper may sit among them.
- **Incomplete full-text coverage.** 31 of the 100 papers are paywalled and 1 was access-denied. Of the 68 open PDFs, 27 are preprints, not versions of record.
- **Database coverage.** The 4,500-row collection has no preserved query strings. ACM, Scopus, and Web of Science returned no records in the documented attempt and are open for repetition.
- **No study-quality appraisal.** The quality split classifies venues. It does not appraise the methods or the evidence of any paper.

11.2 Limitations of AgentShield-LCE

- The lifecycle has not been run. No attack-success rate, utility change, detection rate, overhead, or comparison with another framework is claimed.
- A pass at G1 to G4 means the record contains the named evidence. It does not mean the agent is secure and applies no numerical threshold.
- Controls are design controls. Several host published mechanisms, and hosting them does not transfer those papers' measurements.
- A predicate can be too wide and still be applied consistently. The protocol does not notice a wide check by itself; G3 returns it to Stage 4 only once it is seen.

- A multi-agent observation that is not a tool-trace predicate stays outside the metric table. Unobservable reasoning, host compromise, and chat-only jailbreaks are not scored.
- The lifecycle has no sandbox, no paired-test archive, and no trace archive. A run would be a later study and is not needed to describe the method.

11.3 How a number from this project is to be cited

Cite the file that holds the rows, and say what the number counts. Do not add the October 419 to the 3,059. Do not call the 100 venue-verified. Do not report a metric from the lifecycle as an experimental result. Do not fill a blank DOI, quartile, abstract, or author from memory.

Appendix A. Reconciliation of counts

Table A1. Checks that close each stage

Stage	Reconciliation	Source file
Duplicate removal	4,500 − 1,441 = 3,059; 1,618 papers once + 1,441 papers twice = 3,059	Duplicate_Removal_Count.xlsx
First quality pass	486 + 2,573 = 3,059	publication_quality_filter_counts.json
Cross-verification	486 + 120 = 606; 2,573 − 120 = 2,453; 606 + 2,453 = 3,059	Selected_Excluded_Cross_Verification_Report.md
Title screening	221 + 193 + 192 = 606	Title_Abstract_Screening_and_PDF_Retrieval_Report.md
Abstract screening	232 + 141 + 4 + 36 = 413 = 221 + 192	Same
Final set and reserve	146 at or above 58; 100 final + 46 reserve at or above 58 + 86 reserve below 58 (232 included)	Same
All 606	100 + 132 + 4 + 36 + 141 + 193 = 606	SLR_Record_Flow.xlsx
Full text	41 + 27 = 68; 68 + 31 + 1 = 100	Final_PDF_Verification.xlsx
Crossref status of 3,059	1,134 retrieved + 952 no DOI + 973 not in Crossref = 3,059	publication_quality_filter_counts.json

Appendix B. Earlier documented search (26 and 27 October 2026)

The queries below belong to the earlier bibliography of Section 3.2. Hit counts are as reported by the database for each query. They overlap across queries and cannot be summed to a paper count (the 31 IEEE counts add to 1,943 overlapping hits, which reduced to 952 unique rows).

Table B1. IEEE Xplore queries, 26 October 2026

ID	Hits	Query
IEEE-01	306	("LLM agents" OR "LLM agent" OR "LLM-based agents" OR "LLM-based agent" OR "large language model agents" OR "large language model agent") AND (security OR attack OR defense OR threat OR vulnerability OR adversarial)
IEEE-02	149	"LLM agents" AND security
IEEE-03	0	"large language model agents" AND "security evaluation"
IEEE-04	4	"LLM agents" AND "security assessment"

ID	Hits	Query
IEEE-05	59	("AI agents" OR "autonomous agents") AND ("large language model" OR LLM) AND security AND evaluation
IEEE-06	16	"tool-augmented" AND (LLM OR "language model") AND security
IEEE-07	127	("LLM agent" OR "LLM agents" OR "LLM-based agent") AND (attack OR defense)
IEEE-08	251	("LLM agent" OR "LLM agents" OR "large language model") AND "multi-agent" AND (security OR attack OR adversarial OR defense)
IEEE-09	11	"LLM agent" AND "attack surface"
IEEE-10	37	("LLM agents" OR "LLM agent") AND "threat model"
IEEE-11	129	"prompt injection" AND (agent OR agents)
IEEE-12	39	"indirect prompt injection"
IEEE-13	1	"agent hijacking"
IEEE-14	13	"memory poisoning" AND (LLM OR agent OR agents)
IEEE-15	6	"tool misuse" AND (LLM OR agent OR agents)
IEEE-16	53	("LLM agent" OR "LLM agents") AND (vulnerability OR vulnerabilities)
IEEE-17	16	("LLM agents" OR "LLM agent" OR "language model agent") AND (backdoor OR poisoning OR "privilege escalation")
IEEE-18	10	("LLM agents" OR "LLM agent") AND ("data leakage" OR "privacy attack" OR "privacy attacks")
IEEE-19	30	("LLM agents" OR "LLM agent") AND "attack success rate"
IEEE-20	236	("LLM agent" OR "LLM agents") AND (benchmark OR "security evaluation")
IEEE-21	100	("LLM agent" OR "LLM agents") AND (defense OR mitigation OR guardrail OR guardrails)
IEEE-22	77	"prompt injection" AND (defense OR detection) AND (agent OR agents)
IEEE-23	12	("systematic review" OR survey) AND security AND ("LLM agent" OR "LLM agents" OR "language model agent" OR "LLM-based agent")
IEEE-24	103	"Model Context Protocol" AND (security OR attack OR vulnerability) AND (agent OR LLM)
IEEE-25	0	("LLM agents" OR "LLM agent" OR "prompt injection") AND (security OR attack) AND (agent OR agents)
IEEE-26	87	("large language model" OR LLM) AND (agent OR agents) AND (security OR attack OR privacy OR injection OR vulnerability OR defense OR threat OR backdoor OR poison)
IEEE-27	30	"prompt injection" AND (agent OR agents OR tool OR tools OR LLM OR "language model")
IEEE-28	5	("tool-using" OR "tool augmented" OR "tool-augmented" OR "tool-using language") AND ("language model" OR LLM) AND (security OR privacy OR attack OR injection)
IEEE-29	4	("large language model" OR LLM) AND (agent OR agents OR agentic) AND (security OR attack OR "prompt injection" OR privacy OR backdoor OR poison OR threat OR vulnerability)
IEEE-30	18	("large language model" OR LLM OR "language model") AND (agent OR agents) AND (security OR attack OR privacy OR "prompt injection" OR backdoor)
IEEE-31	14	("LLM agents" OR "LLM agent" OR "LLM-based agent" OR "language model agent" OR "foundation model based agents" OR "foundation model-based agents") AND (security OR attack OR privacy OR risk OR threat OR defense OR injection)

Table B2. Google Scholar query runs, 27 October 2026 (year filter as stated, pages saved as stated)

Query	Years	Pages saved
"LLM agents" "prompt injection"	2020–2026	0–40
"indirect prompt injection" (agent OR agents)	2020–2026	0–70
"web agent" "prompt injection"	2020–2026	0–70

Query	Years	Pages saved
"LLM agents" (defense OR mitigation OR benchmark OR safeguard)	2020–2026	0–70
"multi-agent" (LLM OR "language model") (security OR privacy OR backdoor OR injection)	2020–2026	0–60
"model context protocol" (security OR attack OR injection) (agent OR LLM)	2020–2026	0–50
(backdoor OR poisoning OR "memory poisoning") ("LLM agent" OR "LLM agents" OR "language model agent")	2020–2026	0–50
("tool-using" OR "tool-integrated" OR "tool calling") LLM agent security	2020–2026	0–50
("computer-use agent" OR "computer use agent") security OR prompt injection	2020–2026	0–50
("large language model" OR LLM) (agent OR agents) security	2020–2024	0–10, then no results
"prompt injection" (agent OR agents OR tool)	2020–2024	0–20
"LLM agent" security	2023–2024	0–30
"language model" (chatbot OR assistant) security	2020–2023	0–50; generic chatbot records screened out

The three Crossref queries of 3 November 2026 were “LLM agent security benchmark prompt injection”, “MCP tool LLM agent security”, and “multi-agent LLM security defense”.